

Projetos com Frameworks Front-end

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br

Sumário

- Introdução aos Frameworks Front-end;
- Framework × Biblioteca;
- React, Vue, Angular e Next.js
- Comparação entre Frameworks
- Criação e estrutura de projetos
- Git e versionamento
- Atividade prática

Introdução aos Frameworks Front-end

- Um framework front-end é um conjunto de ferramentas, bibliotecas e convenções que padronizam o desenvolvimento de interfaces web. Ele fornece uma estrutura pré-definida, acelerando a criação de aplicações complexas."
- Comparação com desenvolvimento puro (Vanilla JS):
 - Sem framework: Código manual, difícil manutenção, repetição.
 - Com framework: Componentes reutilizáveis, estado gerenciado, atualizações eficientes.

Diferenças entre Framework e Biblioteca

- Framework
 - Controla o fluxo (inversão de controle)
 - Exige estrutura definida
 - Exemplos: Angular, Vue
- Biblioteca
 - Você controla quando chamar
 - Flexível, sem imposições
 - Exemplos: React, jQuery
- Exemplo prático:
 - **Biblioteca:** Você chama ReactDOM.render() quando quiser.
 - **Framework:** Angular decide quando renderizar componentes.

Por que Utilizar um Framework?

- Produtividade Aumentada: Evita reinventar a roda com soluções prontas para roteamento, estado e renderização.
- Melhores Práticas: Código organizado em componentes, seguindo padrões reconhecidos.
- Manutenção Facilitada: Virtual DOM (React), Change Detection (Angular), e otimizações internas.
- Comunidade e Suporte: Documentação extensa, plugins, e soluções prontas para problemas comuns.

Exemplos de Frameworks

- **React:**

- Criado pelo Facebook, não é um Framework, mas sim uma **biblioteca** JavaScript para construir interfaces de usuário. Embora seja frequentemente chamado de "framework", tecnicamente é uma biblioteca que permite a criação de componentes reutilizáveis.

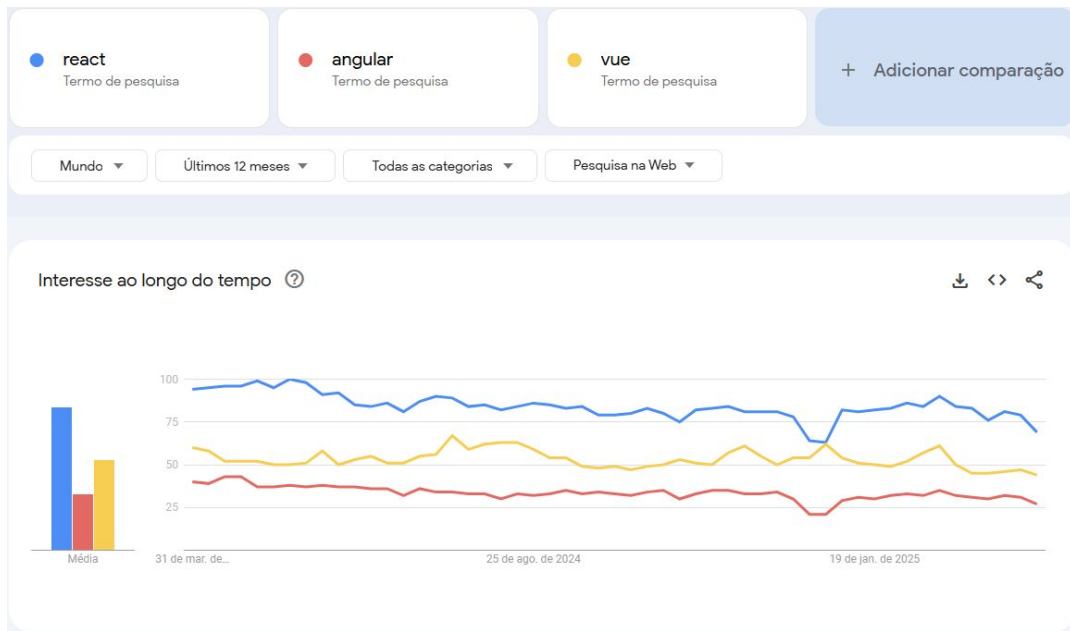
- **Angular:**

- Desenvolvido pelo Google, é um framework completo que oferece soluções robustas para o desenvolvimento de aplicações web de página única (SPA).

- **Vue.js:**

- Um framework progressivo que é fácil de adaptar à medida que as necessidades da aplicação crescem.

Exemplos de Frameworks



Fonte: <https://trends.google.com.br/>

Características dos Frameworks Front-end

- Estrutura de Código Organizada
 - Frameworks front-end oferecem uma estrutura clara, separando HTML, CSS e JavaScript para facilitar manutenção e escalabilidade. Muitos usam componentes, permitindo reutilização de partes da interface.
- Componentização
 - Componentes são independentes e reutilizáveis, encapsulando lógica e apresentação. Isso acelera o desenvolvimento e melhora a manutenção, já que mudanças em um componente não afetam outros.
- Programação Reativa
 - Frameworks como React, Vue e Angular atualizam a UI automaticamente com mudanças no estado da aplicação, eliminando a necessidade de manipulação manual do DOM.

Características dos Frameworks Front-end

- Ferramentas de Build e Bundling
- Frameworks incluem ferramentas para minificar, transpilar e combinar arquivos, melhorando desempenho e compatibilidade com navegadores.
- Sistema de Rotas
- Frameworks permitem criar SPAs (Single Page Applications) com navegação suave e carregamento rápido de conteúdo, sem recarregar a página inteira.
- Integração com APIs
- Facilitam chamadas assíncronas a serviços externos e sincronização de dados entre a UI e APIs, com bibliotecas para gerenciamento de estado.

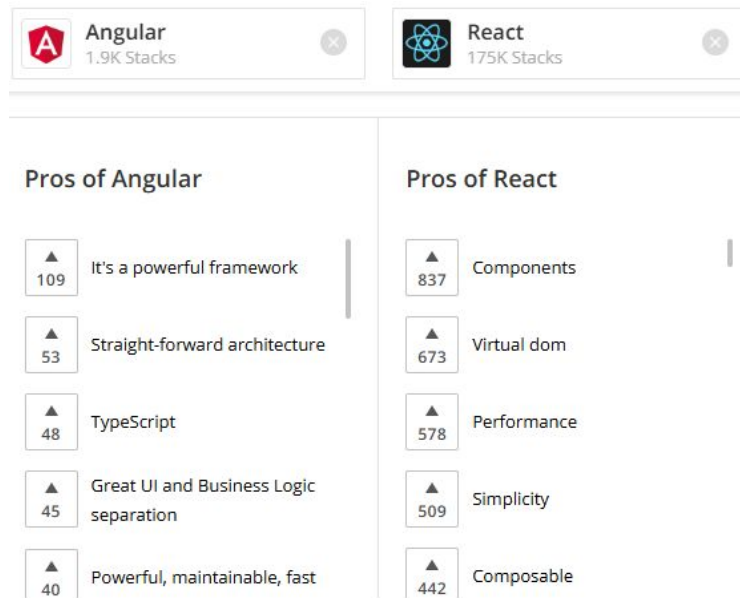
Características dos Frameworks Front-end

- Documentação e Comunidade
- Frameworks populares têm documentação clara e comunidades ativas, com tutoriais, plugins e soluções para problemas comuns.
- Padrões de Design e Acessibilidade
- Promovem boas práticas de design e acessibilidade, com componentes pré-construídos que seguem diretrizes inclusivas.
- Suporte a Testes
- Incluem ferramentas para testes unitários e de integração, garantindo confiabilidade e robustez da aplicação.

Comparação entre Frameworks

A escolha do framework certo para o desenvolvimento de uma aplicação web é uma decisão fundamental que pode impactar diretamente o desempenho, a escalabilidade, a manutenção e a experiência do usuário.

Cada framework tem suas vantagens e desvantagens, e a decisão deve levar em conta fatores como complexidade do projeto, curva de aprendizado, desempenho e suporte da comunidade.



Fonte: <https://stackshare.io/stackups>

Introdução ao React

A biblioteca React, desenvolvida pelo Facebook em 2013, é uma das ferramentas mais populares para criação de Web Apps e interfaces de usuário dinâmicas e eficientes.

Para trabalhar com React, é necessário ter conhecimento prévio em HTML e JavaScript, já que a biblioteca se baseia nesses fundamentos para construir componentes reutilizáveis e interfaces interativas.

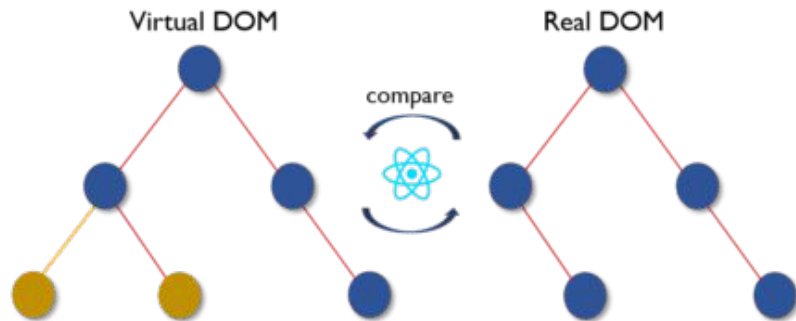
Sua arquitetura baseada em componentes e o uso do Virtual DOM permitem aplicações rápidas e escaláveis, consolidando o React como uma das principais escolhas para desenvolvimento front-end moderno.

Conceitos Fundamentais do React

- Hooks: `useState` e `useEffect`
 - `useState`: Gerencia o estado de um componente funcional.
 - `useEffect`: Lida com efeitos colaterais como chamadas de API.
- JSX: Diferenças entre HTML e React
 - Usa `{}` para expressões JavaScript.
 - Atributos são escritos em `camelCase` (`className` em vez de `class`).
 - Tags precisam ser sempre fechadas (`<img />`).
- Gerenciamento de Estado
 - Context API: Simples e direto, útil para estados menores.
 - Redux: Indicado para estados mais complexos e globalmente compartilhados.

Definição de DOM (Document Object Model)

- O DOM é uma representação em árvore da estrutura de uma página web, permitindo que o JavaScript altere seu conteúdo. O Virtual DOM (usado no React) é uma cópia mais rápida desse modelo. Quando algo muda, o React primeiro atualiza essa cópia, compara com o DOM real e só aplica as diferenças, tornando tudo mais rápido e eficiente.



ANGULAR

Criando um Projeto Angular

- Requisitos
 - Node.js instalado
 - Programação orientada a objetos (POO). Referência:
<https://angular.dev/cli/generate/class#>
- Destaques do Framework
 - Framework completo: Roteamento, HTTP client, injeção de dependências.
 - TypeScript nativo: Melhor suporte a tipos e escalabilidade.
 - Arquitetura MVC: Separação clara de responsabilidades.
 - CLI poderosa: Geração automática de componentes, serviços, etc.
 - Performance: Change Detection eficiente.

Conceitos Fundamentais do Angular

- Componentes: Estrutura com @Component (HTML + CSS + TypeScript).
- Módulos (@NgModule): Organização do app em blocos funcionais.
- Serviços: Lógica reutilizável com @Injectable.
- Data Binding: [(ngModel)] (two-way binding), {{ }} (interpolação).
- Injeção de Dependência: Hierarquia de providers.
- Roteamento: RouterModule para navegação entre views.

Criando um Projeto Angular

- # Passo 1: Instalar o Angular CLI globalmente
 - `npm install -g @angular/cli`
- # Passo 2: Criar um novo projeto Angular
 - `ng new meu-app-angular`
- # Passo 3: Entrar na pasta do projeto criado
 - `cd meu-app-angular`
- # Passo 4: Abrir o VS Code na pasta do projeto (se ainda não estiver aberto)
 - `code .`
- # Passo 5: Iniciar o servidor de desenvolvimento
 - `ng serve`

Angular CLI (Command Line Interface)

É uma ferramenta de linha de comando que ajuda a criar, gerenciar e construir projetos Angular de forma mais eficiente.

Criando um Projeto Angular



Hello, meu-app-angular

Congratulations! Your app is running. 🎉

[Explore the Docs](#)

[Learn with Tutorials](#)

[CLI Docs](#)

[Angular Language Service](#)

[Angular DevTools](#)



```
Server bundles
Initial chunk files | Names | Raw size
polyfills.server.mjs | polyfills.server | 570.97 kB |
main.server.mjs | main.server | 24.55 kB |
server.mjs | server | 1.86 kB |

Application bundle generation complete. [4.205 seconds]

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request
→ Local: http://localhost:4200/
→ press h + enter to show help
```

Fonte: Elaboração própria.

Fonte: Elaboração própria.

Criando um Projeto Angular

- Pasta node_modules
 - Contém os pacotes instalados (bibliotecas JavaScript e dependências do Angular). Use npm install (ou npm i) para instalar as dependências do projeto.
- Pasta public
 - Contém arquivos estáticos, como HTML, JSON, imagens e outros recursos acessíveis publicamente.
- Pasta src
 - É o diretório principal do código fonte do projeto Angular, onde estão os componentes, serviços, módulos e outros arquivos essenciais.
- A pasta .angular é gerada pelo Angular CLI para armazenar cache e configurações temporárias de build, enquanto a pasta .vscode contém configurações específicas do VS Code, como definições de depuração e extensões recomendadas para o projeto. Ambas são criadas automaticamente e geralmente ignoradas pelo Git.

```
> .angular
> .vscode
> node_modules
> public
▼ src
  > app
    <> index.html
    TS main.server.ts
    TS main.ts
    TS server.ts
    # styles.css
  .editorconfig
  .gitignore
  {} angular.json
  {} package-lock.json
  {} package.json
  ⓘ README.md
  {} tsconfig.app.json
  TS tsconfig.json
  {} tsconfig.spec.json
```

Fonte: Elaboração própria.

Criando um Projeto Angular

- O `.gitignore` especifica quais arquivos e diretórios (como `node_modules` e arquivos de ambiente) devem ser ignorados pelo Git. O `package.json` e `package-lock.json` gerenciam as dependências e scripts do projeto. O `.editorconfig` padroniza formatação de código entre editores, enquanto o `README.md` documenta informações essenciais sobre o projeto.
- O `angular.json` é o arquivo de configuração principal do Angular, definindo estruturas de build, testes e estilos globais. Já os arquivos `tsconfig.json`, `tsconfig.app.json` e `tsconfig.spec.json` configuram o TypeScript: o primeiro define as regras base para todo o projeto, o segundo ajusta configurações específicas para a aplicação principal, e o terceiro para os testes.

```
> .angular
> .vscode
> node_modules
> public
▼ src
  > app
    <> index.html
    TS main.server.ts
    TS main.ts
    TS server.ts
    # styles.css
  ⚙ .editorconfig
  <> .gitignore
  {} angular.json
  {} package-lock.json
  {} package.json
  ⓘ README.md
  {} tsconfig.app.json
  TS tsconfig.json
  {} tsconfig.spec.json
```

Fonte: Elaboração própria.

Criando um Projeto Angular

- Pasta app
 - Contém os componentes, módulos, serviços e outros arquivos principais da aplicação Angular.
- index.html
 - Ponto de entrada HTML da aplicação Angular, onde o componente raiz é renderizado via seletor <app-root>.
- [main.server.ts](#)
 - Ponto de entrada para o Angular Universal (renderização server-side).
- [main.ts](#)
 - Arquivo principal que inicializa o módulo raiz (AppModule) e renderiza a aplicação no DOM.
- [server.ts](#)
 - Configuração do servidor para Angular Universal (SSR - Server-Side Rendering).
- styles.css
 - Arquivo de estilos globais que afeta toda a aplicação Angular.

```
> .angular
> .vscode
> node_modules
> public
▼ src
  > app
    <> index.html
    TS main.server.ts
    TS main.ts
    TS server.ts
    # styles.css
  .editorconfig
  .gitignore
  {} angular.json
  {} package-lock.json
  {} package.json
  ⓘ README.md
  {} tsconfig.app.json
  TS tsconfig.json
  {} tsconfig.spec.json
```

Fonte: Elaboração própria.

VUE

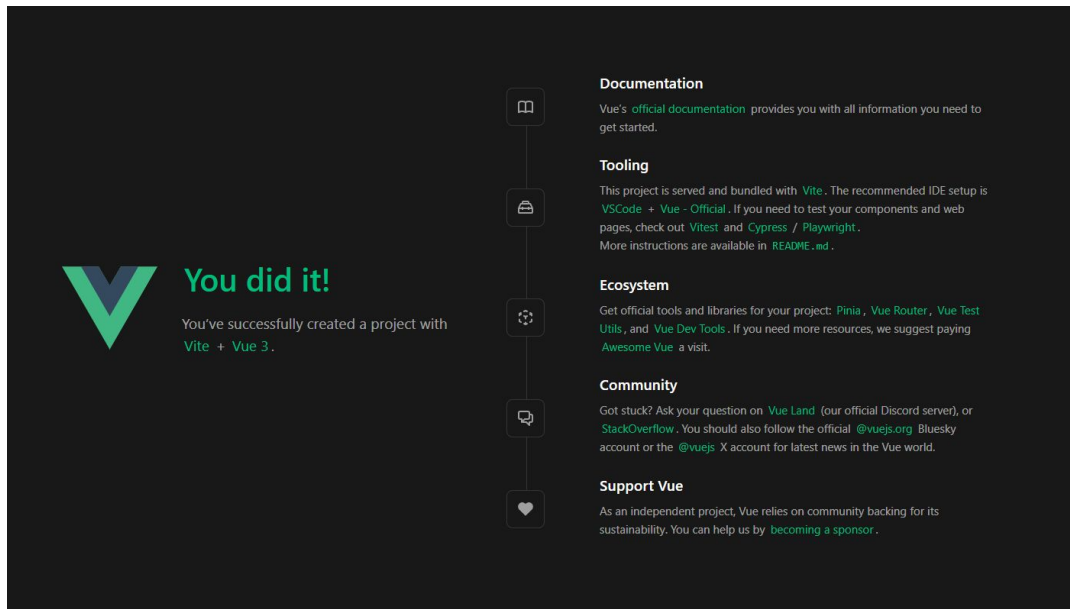
Criando um Projeto Vue

- Requisitos
 - Node.js instalado
 - Conhecimento em JavaScript/TypeScript
 - Programação reativa e baseada em componentes. Referência:
<https://vuejs.org/guide/introduction.html>
- Destaques do Framework
 - Progressivo: Pode ser adotado gradualmente (desde pequenas partes até SPAs complexas).
 - Reatividade eficiente: Sistema de reatividade automático e performático.
 - Single-File Components (SFC): HTML, CSS e JS em um único arquivo .vue (organização intuitiva).
 - Curva de aprendizado suave: Sintaxe acessível para iniciantes, mas poderosa para avançados.
 - Performance otimizada: Virtual DOM eficiente e tamanho reduzido.

Criando um Projeto Vue

- # Passo 1: Criar o projeto Vue
 - `npm create vue@latest`
- # Passo 2: Navegar para a pasta do projeto
 - `cd meu-projeto-vue`
- # Passo 3: Instalar as dependências
 - `npm install`
- # Passo 4: Abrir o VS Code na pasta do projeto (se ainda não estiver aberto)
 - `code .`
- # Passo 5: Iniciar o servidor de desenvolvimento
 - `npm run dev`

Criando um Projeto Vue



Fonte: Elaboração própria.

```
> vue-project@0.0.0 dev
> vite

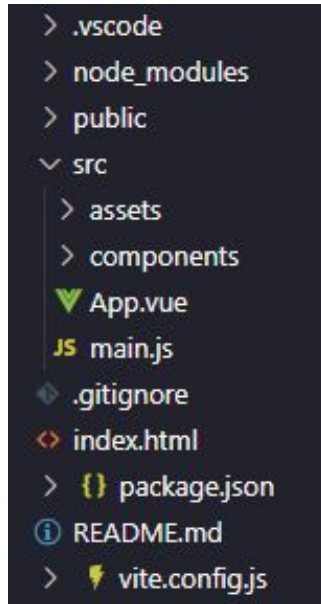
VITE v6.2.6 ready in 1445 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window
→ Vue DevTools: Press Alt(⌘)⇧Shift(⌥)⌘ in App to toggle the Vue DevTools
→ press h + enter to show help
```

Fonte: Elaboração própria.

Criando um Projeto Vue

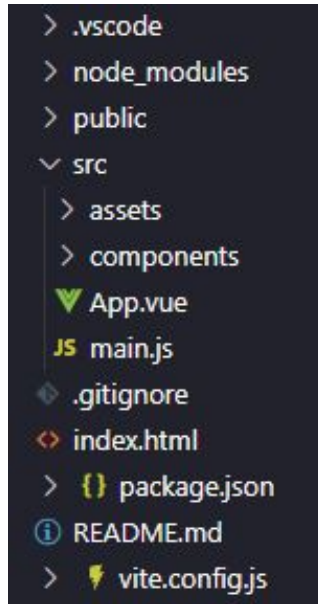
- Pasta `node_modules`
 - Contém os pacotes instalados (bibliotecas JavaScript e dependências do Vue). Use `npm install` (ou `npm i`) para instalar as dependências do projeto.
- Pasta `public`
 - Arquivos estáticos que não passam pelo build do Vite (ex.: `favicon.ico`, `robots.txt`). São copiados diretamente para a pasta final de produção.
- Pasta `src`
 - É o diretório principal do código fonte do projeto Vue, onde estão os componentes, serviços e outros arquivos essenciais.
- A pasta `.vscode` contém configurações específicas do VS Code, como definições de depuração e extensões recomendadas para o projeto. O `.gitignore` especifica quais arquivos e diretórios devem ser ignorados pelo Git. O `package.json` e `package-lock.json` gerenciam as dependências e scripts do projeto. O `vite.config.js` contém as Configurações do Vite (build, plugins, proxies, etc.).



Fonte: Elaboração própria.

Criando um Projeto Vue

- Pasta assets
 - Imagens, fonts, CSS global, etc. (arquivos estáticos que são processados pelo Vite).
- Pasta components
 - Componentes Vue reutilizáveis (ex.: Button.vue, Header.vue).
- App.vue
 - Componente raiz da aplicação. Define a estrutura inicial (HTML, CSS, JS) e importa outros componentes.
- [main.js](#)
 - Ponto de entrada do Vue. Monta o app no DOM e configura plugins globais.
- index.html
 - Único HTML da SPA. Contém a div #app onde Vue é injetado. O Vite injeta automaticamente os scripts do build.



Fonte: Elaboração própria.

NEXT

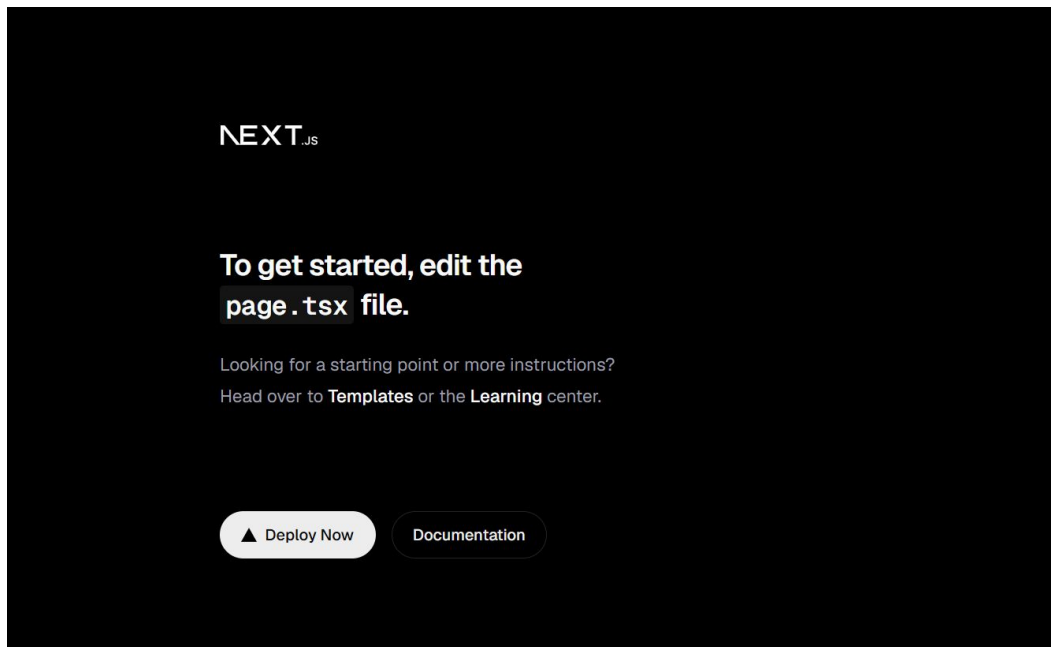
Criando um Projeto Next

- Next.js é um framework baseado em React para desenvolvimento de aplicações Web modernas e full-stack.
- Ele adiciona recursos que não fazem parte diretamente do React, como:
 - Roteamento baseado em arquivos;
 - Renderização no servidor;
 - Server Components;
 - Otimização de imagens e fontes;
 - Gerenciamento de páginas e layouts;
 - APIs e recursos de backend;
 - Otimizações para desempenho e SEO.

Criando um Projeto Next

- # Passo 1: Criar o projeto Next
 - `npx create-next-app@latest meu-projeto`
- # Passo 2: Navegar para a pasta do projeto
 - `cd meu-projeto`
- # Passo 3: Abrir o VS Code na pasta do projeto (se ainda não estiver aberto)
 - `code .`
- # Passo 4: Iniciar o servidor de desenvolvimento
 - `npm run dev`

Criando um Projeto Vue



Fonte: Elaboração própria.

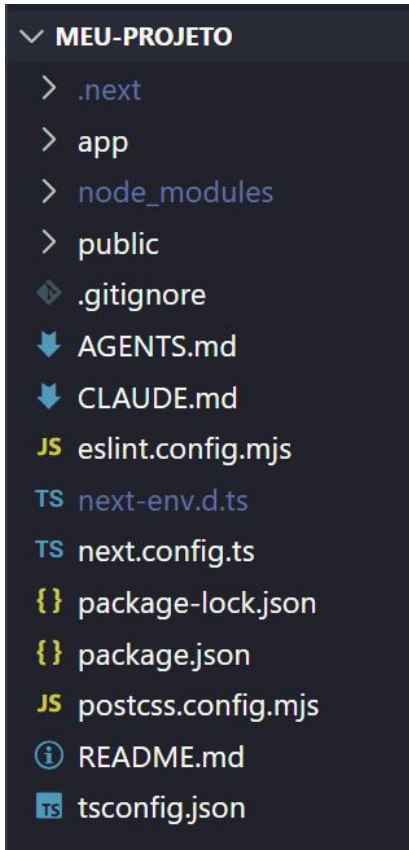
```
PS C:\Users\Acer\Desktop\meu-projeto> npm run dev
stead.
▲ Next.js 16.3.1 (Turbopack)
- Local:      http://localhost:3001
- Network:    http://192.168.56.1:3001
✓ Ready in 707ms
✓ Running next.config.ts took 53ms
✗ Another next dev server is already running.

- Local:      http://localhost:3000
```

Fonte: Elaboração própria.

Criando um Projeto Next

- Pasta `node_modules`
 - Contém os pacotes instalados (bibliotecas JavaScript e dependências do Vue). Use `npm install` (ou `npm i`) para instalar as dependências do projeto.
- Pasta `public`
 - Arquivos estáticos que não passam pelo build do Vite (ex.: `favicon.ico`, `robots.txt`). São copiados diretamente para a pasta final de produção.
- Pasta `app`
 - É o diretório principal do código da aplicação quando utilizado o App Router do Next.js. Contém as páginas, layouts, estilos globais e outros componentes relacionados à estrutura da aplicação. Arquivos `page.js` definem páginas e a organização das pastas determina as rotas.

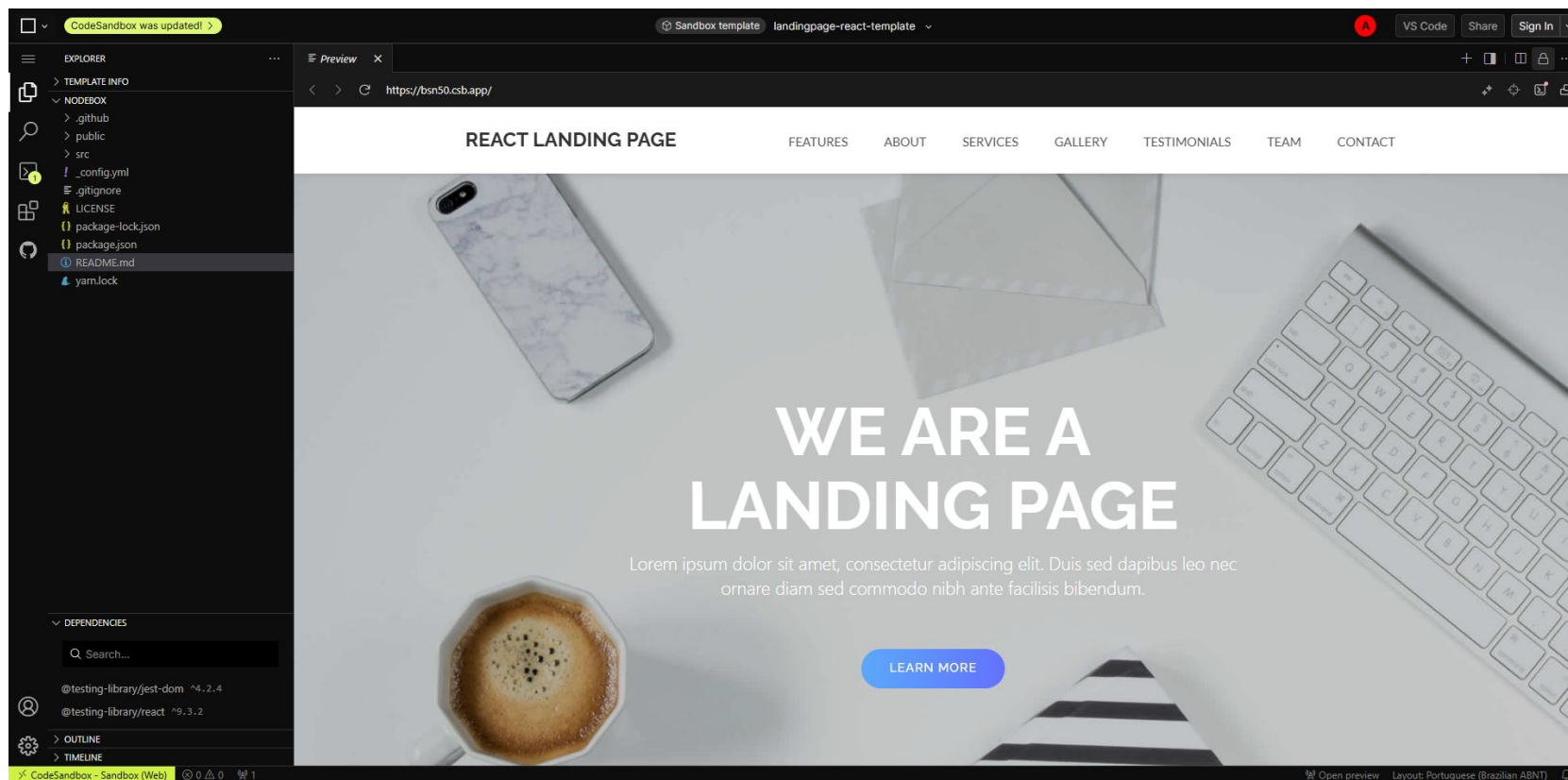


Fonte: Elaboração própria.

IMPORTANDO PROJETOS

Buscando Projetos por Frameworks

- Encontrar projetos modelos, desenvolvidos com um framework desejado, pode acelerar muito o desenvolvimento da aplicação. Muitas vezes não é necessário começar do zero, pois, a comunidade open source oferece centenas de projetos gratuitos, personalizáveis e com boas práticas de código.
- Ferramentas para pesquisar projetos:
- GitHub - [Repository search](#) *Utilizar o comando git clone <url>
- Vercel - [Busca por Templates](#) *Fazer o download apenas de uma parte do repositório
- CodeSandbox - [Template Search](#)



Atividade

Desenvolva, em grupo, quatro projetos Web sobre o mesmo tema, utilizando React, Vue, Angular e Next.js. Cada projeto deverá apresentar uma página funcional, responsiva e organizada, utilizando componentes e os recursos básicos da tecnologia escolhida.

Atividade

Durante o desenvolvimento, os projetos deverão ser versionados com Git e publicados no GitHub, mantendo um histórico de commits que registre a evolução da aplicação. Ao final, organize cada projeto em seu respectivo repositório e elabore uma breve comparação entre as quatro tecnologias, destacando as principais diferenças encontradas durante o desenvolvimento.

Atividade

- As entregas deverão ser:
 - Projeto 01: React;
 - Projeto 02: Vue;
 - Projeto 03: Angular;
 - Projeto 04: Next.js;
 - Projeto 05: Uma cópia de um projeto a partir de um repositório.

Referências

1. SOUZA, Natan. ****Bootstrap 4: conheça a biblioteca front-end mais utilizada no mundo.**** São Paulo: Casa do Código, 2018. E-book. Disponível em: <https://plataforma.bvirtual.com.br>.
2. MACHADO, Kheronn Khennedy. ****Angular 11 e Firebase: construindo uma aplicação integrada com a plataforma do Google.**** São Paulo: Casa do Código, 2021. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 13 maio 2025.
3. EIS, Diego. ****Guia Front-end: o caminho das pedras para ser um dev front-end.**** São Paulo: Casa do Código, 2015. E-book. Disponível em: <https://plataforma.bvirtual.com.br>.
4. GONÇALVES, Edson. ****Desenvolvendo aplicações Web com JSP, Servlets, JavaServer Faces, Hibernate, EJB 3 Persistence e Ajax.**** Rio de Janeiro: Ciência Moderna, c2007.

Referências

5. HARTCOPP, Patrícia Ferreira. ****Métrica Web.**** São Paulo: Contentus, 2020. E-book (94 p.). Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/185191>. Acesso em: 30 abr. 2024.
6. NIEDERAUER, Juliano. ****Desenvolvendo Websites com PHP: aprenda a criar Websites dinâmicos e interativos com PHP e banco de dados.**** 3. ed. São Paulo: Novatec, 2017.
7. PREECE, J.; ROGERS, Y.; SHARP, H. ****Design de Interação: além da interação Homem-Computador.**** 3. ed. Porto Alegre: Bookman, 2013.
8. SOUSA, Roque Fernando Marcos. ****Canvas HTML 5: composição gráfica e interatividade na Web.**** Rio de Janeiro: Brasport, 2014. E-book (194 p.). Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/160686>. Acesso em: 22 jun. 2024.

Projetos com Frameworks Front-end

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br